

There are a few important points to be noted in the code given above.

- `sys_call_table` is an array of size `__NR_syscall_max+1`. `__NR_syscall` is defined as `macro #define __NR_syscall_max 547` (may not be the latest) in the `include/generated/asm-offsets.h` header file generated at the time of kernel build.
- Type of `sys_call_table` array is `sys_call_ptr_t`. This is `typedef void (*sys_call_ptr_t)(void);`
- Initially all the elements of the array that contain pointers to the system call handlers point to the `sys_ni_syscall`. The `sys_ni_syscall` function only returns `errno -ENOSYS`.
- The most important thing to note is `#include <asm/syscalls_64.h>` in the last line.
- `<asm/syscalls_64.h>` header file will be generated by `arch/x86/entry/syscalls/syscall_64.tbl` through a script `arch/x86/entry/syscalls/syscalltbl.sh`.
- The generated header file `asm/syscalls_64.h` will contain macro definitions similar to what is given below:

```
__SYSCALL_COMMON(0, sys_read, sys_read)
__SYSCALL_COMMON(1, sys_write, sys_write)
__SYSCALL_COMMON(2, sys_open, sys_open)
__SYSCALL_COMMON(3, sys_close, sys_close)
__SYSCALL_COMMON(5, sys_newfstat, sys_newfstat)
...
...
...
```

- The `SYSCALL_COMMON` macro is defined in `arch/x86/entry/syscall_64.c` as below:

```
#define __SYSCALL_COMMON(nr, sym, compat) __SYSCALL_64(nr, sym, compat)
#define __SYSCALL_64(nr, sym, compat) [nr] = sym,...
```

- Finally, after expansion of the macro the `sys_call_table` will look like this:

```
asmlinkage const sys_call_ptr_t sys_call_table[__NR_syscall_max+1] = {
    [0 ... __NR_syscall_max] = &sys_ni_syscall,
    [0] = sys_read,
    [1] = sys_write,
    [2] = sys_open,
    ...
    ...
    ...
};
```

- Now each array index of `sys_call_table`, where the index is equal to system call number, contains a corresponding

system call handler address. Whereas the array indexes corresponding to non-implemented system calls will have the address of `sys_ni_syscall`, which returns `-ENOSYS`.

Execution of system call handler

We now understand that the kernel maintains a `sys_call_table` array that contains addresses of each system call handler function. But how are these system call handler functions invoked when user space applications execute a syscall instruction? In order to understand this, check the description in the box below.

From the Intel manual

SYSCALL invokes an OS system-call handler at privilege level 0. It does so by loading RIP from the `IA32_LSTAR` MSR.

`IA32_LSTAR_MSR` is a model-specific register.

A model-specific register (MSR) is any of various control registers in the x86 instruction set used for debugging, program execution tracing, computer performance monitoring, and toggling certain CPU features.

Reading and writing to these registers is handled by the `rdmsr` and `wrmsr` instructions, respectively. As these are privileged instructions, they can be executed only by the operating system.

It's now clear that the kernel must write `IA32_LSTAR_MSR` with the address of kernel entry code that needs to be executed when a user space application triggers a system call request (by executing syscall instruction). This `IA32_LSTAR_MSR` is written with the address of system call entry point code during the kernel initialisation. Here are some of the important code snippets for system call handler execution by the Linux kernel. Check the code comments marked in red to understand the details about the implementation.

```
/*MSR_LSTAR is written with the system call handling entry
code
*entry_SYSCALL_64 which is defined in arch/x86/entry/
entry_64.S.
*This means after syscall instruction in user mode,
*entry_SYSCALL_64 will be executed in kernel code.
*/
```

```
/arch/x86/kernel/cpu/common.c
void syscall_init(void)
{
    wrmsr(MSR_STAR, 0, ((USER32_CS << 16) | __KERNEL_CS));
```